Summation-based Private Segmented Membership Test from Fully Homomorphic Encryption

***Abstract*—In many real-world scenarios, there are cases where a client wishes to check if a data element s/he holds is included in a set segmented across a large number of data holders. To protect user privacy, the client's query and the data holders' sets should remain encrypted throughout the whole process. Prior work on Private Set Intersection (PSI), Multi-Party PSI (MPSI), and Private Membership Test (PMT) falls short in this scenario in many ways. They require data holders to possess the sets in plaintext forms, incur prohibitively high latency for aggregating results from a large number of data holders, leak the information about the party holding the intersection element, or induce a significant false positive rate.**

This paper introduces the primitive of a Private Segmented Membership Test (PSMT). We give a basic construction of PSMT using approximate-arithmetic homomorphic encryption and show how to overcome existing challenges to construct a PSMT protocol without leaking information about the party holding the intersection element or false positives for a large number of data holders. Our novel approach is superior to existing state-of-the-art approaches in scalability with regard to the number of supported data holders. This is enabled by a novel summation-based homomorphic membership check rather than a product-based one as well as various novel ideas addressing technical challenges. Such scalability is essential for highly distributed datasets. Our PSMT protocol supports many more parties (up to 4096 in experimental comparisons) compared to prior related work that supports only around 100 efficiently. Our experimental evaluation shows that our method's aggregation of results from data holders can run in 92.5s for 1024 data holders and a set size of 2^{25}, and our method's overhead increases very slowly when the number of data holders increases due to the efficiency of summation-based homomorphic evaluation. We also compare our PSMT protocol to other state-of-the-art PSI and MPSI protocols and discuss our improvements in usability with a better privacy model and a larger number of parties.

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Privacy concerns often limit the collaboration of many entities in cases where each of them has private data that must be shared for joint usage. In many cases, the private data is generated and stored in a distributed manner, and methods of sharing data privately in such scenarios open avenues for new applications. There are many real-world scenarios where this problem needs to be solved.

Federal tax authorities want to learn whether any suspected tax evaders maintain accounts not only in domestic but also in foreign banks that might be under scrutiny and, only if so, obtain their account records and details. The banks' location in different jurisdictions prohibits the wholesale disclosure of account holders, and the tax authorities clearly cannot openly divulge their list of suspects. Institutions under such scrutiny want to collaborate anonymously to avoid any bad publicity for being linked to a tax fraud operation. In many cases, these institutions are willing to collaborate with the tax authorities [28], and they themselves also wish to exercise rigorous scrutiny when extending loans to new and existing customers to mitigate potential risks. Unfortunately, many financial privacy laws [40] prohibit banks from revealing customer data to third parties without consent. There are currently over 4,700 FDIC-insured banks in the United States only. When a customer applies for loans or any kind of financial assistance within one of them, collaboration and sharing of existing fraud lists can make the decision process significantly more trustworthy. In such cases, banks do not wish to share their specific fraud lists, and they do not even wish to disclose whether the queried customer is on their own fraud list due to various privacy and legal concerns. Such a secure membership query scenario could extend to credit card companies, tax-collection agencies, and similar entities, necessitating the involvement of many parties in the decision-making process. A data-sharing protocol that allows queriers to learn only whether a queried entity exists in distributed fraud lists would help such institutions examine a person's credibility beyond the nation.

Many government agencies (e.g., FBI, CIA) maintain sensitive lists of secret agents or watchlists distributed across its multiple divisions and branches. Data sharing for identity verification, background checks, security clearance, or watchlist screening requires many government entities to work together. For example, a querier may want to verify an individual's presence in these entities' databases without inferring any membership details to verify the identity or the background. Doing so over distributed databases is nontrivial due to the challenges of sharing personally identifiable information (PII). Therefore, it is desirable and even imperative that the records be stored in encrypted forms across a large number of distributed servers such that each record is under strict security/privacy protection (e.g., DHS Use Cases [41]).

A similar problem exists in the auto insurance industry. If an insurance company is able to learn the complete driving violation lists of a customer, e.g., those held by other companies or the Department of Moving Vehicles, they can minimize risks. An analogous problem exists in the medical insurance industry, where one's medical insurance records are stored in a highly distributed manner.

In all the examples above, we face a privacy problem where the scale of entities involved in data-sharing applications can be substantial (hundreds of thousands [47] in the tax fraud).

Furthermore, datasets are updated frequently, and regulations force such datasets containing PII to be stored and operated with strong data security guarantees These examples all underscore the growing need for privacy-preserving set intersection protocols with a substantial number of dataset holders. There is a need to perform queries by testing the membership of a client's element within distributed datasets without leaking information about which set the intersection came from. We call this property *provenance privacy*, which refers to the confidentiality of from which party the intersection comes from. These datasets are updated frequently and held by multiple distinct parties, which should be strictly protected to ensure individual privacy. For this purpose, it is desirable to store and use all the records in the encrypted form, and it requires the computation on data to be carried out in the encrypted domain to safeguard against data breach attacks or insider threats. This scenario is different from those of traditional PSI, MPSI, or PMT in multiple dimensions. We term this problem *Private Segmented Membership Test* (PSMT) and provide its formal definition.

Some approaches, like private set intersection (PSI), fall short in numerous ways for the problems described above. PSI protocols enable two parties (referred to as *receiver* and *sender* hereafter by following the convention [37]) to compute the intersection of their sets without revealing any additional information beyond the intersection. The receiver obtains the intersection results (either the intersection set or only whether an intersection exists) from the sender in PSI protocols. Existing PSI-based protocols can be employed to solve the PSMT problem; however, they require dataset holders to access the elements of their sets in plaintext format to enable various optimizations to compute polynomial interpolation, and they do not scale well with the number of parties. In many situations where the database is encrypted and contains highly sensitive information such as medical, financial, or criminal records, privacy concerns may prevent the sender from accessing the datasets in plaintext. PSI is, therefore, not suitable for scenarios where databases are encrypted. Many generic multiparty-PSI (MPSI) based protocols [9], [32], [39] are also less suitable for efficiently solving PSMT due to a high number of interactions (in OT-based), lack of privacy of datasets held by senders, high computation or communication runtime, bandwidth or storage needs. There also exists a variant of two-party PSI that deals with client sets having only a single element, and it is known as the Private Membership Test (PMT) [48]. PMT protocols also require an unencrypted database, and these protocols generally employ filters for representing the database, resulting in high false positives. Furthermore, they are not designed to be optimized for multiparty applications, and using existing PMT protocols for multi-parties breaches the provenance privacy of the senders.

Fully Homomorphic Encryption (FHE) is a widely used building block for constructing protocols for PSI and similar functionality, as it only requires one round of communication. There exist limitations for PSI protocols based on FHE [11], [12], [21] that are challenging to resolve for the PSMT problem. Theoretically, given a set held by the sender X and receiver query y, existing PSI protocols homomorphically compute a *sender polynomial*, $f(y) = r\prod_{x\in X}(x-y)$ where r is a random multiplicative mask. To use this approach for solving PSMT, each sender would individually calculate their own sender polynomial, after which the multiplicative aggregation of these polynomials would correctly be an encryption of zero if a set held by the sender had an element in the receiver's set and a random nonzero value otherwise. However, doing this for even a moderately large number of senders l (e.g., $l = 64$) or sender polynomials requires FHE parameters that require tolerating $O(log(l))$ *additional* multiplicative depth beyond the depth required for computing the sender polynomial. Furthermore, senders often use partitioning to optimize the computation of sender polynomials by dividing sets into smaller subsets. While this reduces the sender's multiplicative depth during polynomial computation, it increases depth during result aggregation, especially when dealing with a large number of sender polynomials per sender or for multiple senders.

Our PSMT protocol assumes encrypted input for both the receiver's element and sets held by the sender and does not rely on any preprocessing (besides encryption) of the sets beforehand, therefore completely eliminating the need for the senders to access the sets in plaintext. This simplicity enables the protocol to handle frequent updates to the sets without repeating the preprocessing steps. Our protocol's construction is based on FHE, where the receiver uses cheap homomorphic additions to aggregate ciphertexts gathered from multiple senders. The security of our protocol is derived from the post-quantum security of FHE [1], and it does not breach the provenance privacy of the senders to the receiver. We assume a semi-honest model where all parties operate on homomorphically encrypted datasets. In summary, we construct a protocol that can tolerate a large number of senders and is able to easily handle the cases where 1) each sender only possesses their set in encrypted form, and 2) updates may occur frequently.

The contributions of this work are summarized as follows:

- We define the Private Segmented Membership Test (PSMT) problem, which is widely relevant to real-world scenarios. Existing approaches result in various limitations, and we present a novel solution to address them.
- We address the shortcomings of existing PSI and MPSI-based approaches using finite-field FHE for solving PMST, which results from encrypted user data segmented across many senders and high aggregation latency. For the first time, we provide a novel summation-based set intersection protocol with approximate arithmetic FHE that overcomes these limitations.
- For the technical challenges in solving the PSMT problem with our novel solution, including plaintext domain size, function approximation accuracy/latency, and throughput, we provide novel strategies to deal with such issues and achieve good performance even for a large number of senders.

- We implement our method and present an experimental evaluation of our solution to show its significant performance advantage in the case of a large number of senders. Our anonymized source code is available for reproducibility and future research at https://anonymous.4open.science/r/psmt-1323/. We show up to 4-11× performance improvement over previous works.

II. Related Work

A. Private Set Intersection (PSI)

The first PSI protocol was presented based on the Diffie-Hellmann (DH) key agreement scheme [35]. This protocol leveraged the commutative properties of the DH function and offered security against the random oracle model. Its low communication cost continues to serve as a foundation for many modern PSIs. [26] introduced PSI protocols based on oblivious polynomial evaluation (OPE) where sets are represented as polynomials. Additionally, PSI protocols have been constructed using Oblivious Pseudo-Random Functions (OPRFs) [25], garbled circuits [54], and oblivious transfer (OT), and OT-extension [46]. Recently, FHE with post-quantum security started to be used for many PSI protocols for unbalanced set sizes using OPE, where the difference in sets held by the sender and receiver is very high.

PSI between only two parties (receiver and sender) is a well-studied PSI model due to its wide applications in the real world. Several variants of this model exist, where either both parties learn the intersection (mutual PSI) [23] or only one of the parties learns the intersection (one-way PSI). Other variants also allow the computation of different functions on the intersection [23], [44]. Many of these protocols scale to millions of items within seconds and are only slightly slower than the simple but insecure method that exchanges hashed items. Pinkas *et al.* [46] used (1-out-of-n) OT based on [31]. The limitation is that to perform the OT step, the sender needs to access the elements in the bins of the hash table. Furthermore, extending this protocol to a large number of parties requires multiple OPRF evaluations via OT, increasing the communication overhead significantly.

CLR17 [12] protocol, and its improved variants [11], [21] are state-of-the-art FHE-based PSI protocols to the best of our knowledge. The protocols are based on the work presented by [45] and employ the Fan-Vercauteren (B/FV) [24] leveled FHE scheme. The basic protocol in [12] has the sender sample a uniformly random non-zero element r_i and homomorphically compute the intersection polynomial $z_i = r_i \prod_{x \in X}(c_i - x)$ using encrypted receiver's set $(c_1, c_2, \ldots, c_n)$ and the sender's unencrypted elements $x \in X$. z_i is returned to the receiver, who concludes that $y \in X$ iff $z_i = 0$. The receiver only learns the presence of an intersection. The CLR17 protocol applies many optimizations, including receiver and sender side batching using Cuckoo hashing and binning, SIMD (Single Instruction Multiple Data) using FHE, and windowing. Later work [11] used an OPRF preprocessing on the encoded sender set and achieved malicious security. One major limitation of OPRF is that it requires an expensive offline preprocessing of the sender set. [11] also extend the protocol to the labeled setting with arbitrary-length labels. [21] further optimized [11] by including the application of the Paterson-Stockmeyer algorithm for evaluating the intersection circuit in $\mathcal{O}\sqrt{|X|}$ and reducing the communication cost by using extremal postage-stamp bases

These protocols using FHE offer solutions that require a communication complexity of $O(|Y| \log |X|)$ [11], [12] or $O(|Y|(\log \log |X|)^2)$ [21]. The computational complexity of the FHE-based solutions is $O(|X||Y|)$. These protocols must let the sender access the set in plaintext format for the aforementioned optimizations and encodings for creating the polynomial for interpolation. Consequently, the privacy of data points in the sets held by the sender is only protected against the receiver and not against the sender.

Fundamentally, the FHE-based solutions employ zero as a multiplicative annihilator while computing the polynomial $\prod_{x \in X}(y - x)$ homomorphically. This ensures that all the terms not indicating an intersection are annihilated by the value of zero that indicates an intersection during OPE. Adapting these methods to the multi-party scenario drastically increases the multiplicative depth required to obtain the query result. This would also require increasing the ciphertext sizes and the latency of homomorphic operations greatly, losing the scalability completely.

B. Multi-party PSI (MPSI)

Multiparty PSI (MPSI) is a generalization of PSI that refers to the problem where more than two parties wish to compute the intersection of their private sets. This case has fewer efficient solutions compared to the simpler two-party case. Two-party PSI protocols can be extended to multiple parties to handle the MPSI scenario; however, these solutions often lead to privacy and performance issues [53]. Several techniques have been employed to design MPSI, such as circuit-based computations [43], bloom filters [36], OPE [26], and OT and permutation-based hashing [42]. [32] used a technique based on oblivious evaluation of a programmable pseudorandom function (OPPRF) to implement the first MPSI protocol that is highly time-efficient for large amounts of items. However, their time complexity scales quadratically w.r.t the number of parties in the protocol. Chandran *et al.* [9] improves upon [32] in terms of communication and extends it to circuit-based PSI and quorum-PSI. They use Shamir's secret-sharing scheme for a relaxed adversarial power. These protocols either provide the intersection results to all the parties involved or some parties depending on the intersection result and are not based on FHE construction, making their work not comparable to ours.

Recent work by Badrinarayanan *et al.* [2] uses threshold FHE to construct threshold MPSIs, which have sublinear communication complexities with a threshold number that is sublinear in the number of elements in datasets. They use a similar polynomial encoding of each set element as in [12]. [4] provide two MPSI protocols based on bloom filters and threshold homomorphic public-key techniques. Their protocol performs better than previous state-of-the-art in terms of run

time, given that the sets are small and a large number of senders exist.

C. Private Membership Test (PMT)

Private Membership Test (PMT, also called Private Set Inclusion) is a similar problem to PSI, in which a single receiver learns the intersection of their element among several senders. To solve PMT, existing works apply Private Information Retrieval (PIR) based protocols that allow a user to retrieve an item from a database without the database owner learning anything about the item [34], [38]. PSMT problem closely matches the PMT problem. PMT has been thoroughly studied for malware detection for two-party PSI in the past [33], [48]. One naive solution to PMT or PIR with multi-party offering very low latencies would be hashing. However, hashing becomes insecure if the input domain has low entropy, and even with a high entropy input domain, consecutive executions of the hashing-based protocol leak repeated elements.

The PSMT can also be solved by having the client run individual PMT protocols with all participating sender parties using 1-out-of-n OT-based PMTs that output the result in encrypted or secret shared format to the client. The client can then run a secure protocol that computes an XOR of all results privately. However, this approach falls short for PSMT for several reasons. Firstly, it requires the sender parties to access the sets' elements in plaintext to perform the OT and requires extra communication and computation for computing the XOR at the end. Moreover, this method demands the client to run n PMT protocols with n senders, contributing to increased communication complexity and rounds. While OT extension-based protocols such as those described in [46] can reduce communication complexity, they also mandate access to the plaintext sets held by senders. Similarly, any updates in the databases incur huge performance and communication penalties.

Some works have applied the PIR protocols to the PMT problem [19], [48] based on homomorphic encryption and bloom filters. These protocols require a greater amount of communication and induce significantly high false positive rates as clients have to reveal bits of their query and also learn about the server database from the filters. Tamrakar *et al.* [52] propose a carousel method for PMT for solving malware detection based on Trusted Execution Environments (TEEs). The proposed method circulates a cuckoo filter-based dictionary through a trusted application in a way that improves efficiency when processing batched queries. However, TEEs suffer from side-channel attacks [27] and other kinds of hardware-based attacks [10], which have decreased their confidence for use recently.

In summary, PSMT can be addressed using general PMT-solving methods based on PIR, OT, or TEEs, but they only offer partial solutions. The considerable overhead for a large number of senders, a lack of privacy for datasets held by senders, either from the client or the sender(s), along with high latency and low throughput typically associated with PMT protocols render them impractical for efficient PSMT solutions. We compare representative works to PSMT in **??**.

III. Preliminaries

A. Security Definitions

We say that a function $\mu : \mathbb{N} \rightarrow \mathbb{N}$ is negligible (in k) if for every positive polynomial $p(\cdot)$ and all sufficiently large k it holds that $\mu(k) < \frac{1}{p(k)}$. We denote negligible functions by $negl(k)$.

Definition 1 (Computational Indistinguishability): We say that two distributions ensembles $X = \{X_k\}_{k \in N}$ and $Y = \{Y_k\}_{k \in N}$ indexed by security parameter k are computationally indistinguishable if for any probabilistic polynomial time (PPT) algorithm $\mathcal{A}$:

$|Pr[1 \leftarrow \mathcal{A}(X_k)] - Pr[1 \leftarrow \mathcal{A}(Y_k)]| = negl(k)$.

We denote this property by $X \approx_c Y$. We prove the security of our protocol in the *semi-honest model* with *static adversaries* where all protocol participants are assumed to run PPT. We have:

- *Adversaries* - can be any subset of the protocol participants, and we refer to them as *corrupted parties*. We do not consider adversaries outside the protocol as they can be mitigated using standard network security techniques.
- *Static adversaries* - the set of corrupted parties is determined before the execution of the protocol and does not change during the execution. Hence, the honest parties cannot become corrupted and reveal their values during the protocol execution.
- *Semi-honest model* - the corrupted parties follow the protocol honestly, and they can collude to infer information about the honest parties but do not exhibit active malicious behavior and do not deviate from the protocol. While the semi-honest model offers weaker security than the malicious model due to predictable adversaries, it is appropriate in our privacy-preserving setting since it guarantees that there is no unintentional leakage of private information. We note that with appropriate extensions, the protocol can be turned into a maliciously secure model, but it will be much less efficient.

We use the following definition to define security against semi-honest adversaries.

Definition 2 (Semi-Honest Security for Deterministic Functionalities): Let $f : (\{0,1\}^*)^l \rightarrow (\{0,1\}^*)^l$ be a deterministic function of a l-party protocol Π. Here, $f_i(x_1, \ldots, x_l)$ is the i-th component of f. Let $f_{\mathcal{I}}(x_1, \ldots, x_l)$ denote the subsequence of $f_{i_1}(x_1, \ldots, x_l)$, ... , $f_{i_l}(x_1, \ldots, x_l)$, for $\mathcal{I} = \{i_1, \ldots, i_l\} \subset \{1, 2, \ldots, l\}$. We assume that Π computes f. Then, the view of i-th party during an execution of the protocol Π on input $x = (x_1, \ldots, x_l)$ is denoted by $\text{VIEW}^{\Pi}_{\mathcal{I}}(x) = (\mathcal{I}, \text{VIEW}^{\Pi}_{i_1}, \ldots, \text{VIEW}^{\Pi}_{i_l})$. We say that Π privately computes f, if there exists a PPT algorithm denoted by a simulator $\mathcal{S}$ such that for every $\mathcal{I}$:

$\{\mathcal{S}(\mathcal{I}, (x_{i_1}, \ldots, x_{i_l}), f_{\mathcal{I}}(x))\}_{x \in (\{0,1\}^*)^l}$
$\approx_c \{\text{VIEW}^{\Pi}_{\mathcal{I}}(x)\}_{x \in (\{0,1\}^*)^l}$.

The security definition above states that the view of each party in $\mathcal{I}$ can be simulated by only looking at their inputs and

outputs. We assume a semi-honest security model with static adversaries (both senders and receiver). We do not consider membership inference attacks with repeated adaptive queries. Such attacks can be mitigated by rate limiting or OPRFs [11] at the cost of additional overhead from preprocessing. We also do not consider integrity attacks where an adversary may homomorphically modify the sender's set. Securely storing hashes of ciphertexts (e.g., in an isolated storage) would prevent this.

B. Fully Homomorphic Encryption

Fully Homomorphic Encryption (FHE) is a cryptographic primitive that allows computation on encrypted data with post-quantum security. Noise associated with an FHE ciphertext grows corresponding to each homomorphic operation, i.e., additively with additions and multiplicatively with multiplications. The most prominent FHE schemes are BGV [8], B/FV [24], CKKS [15], and Torus-FHE [20]. In practice, FHE schemes are often implemented as Somewhat Homomorphic Encryption (SHE) schemes with a limited multiplicative depth

In this work, we use the CKKS scheme, which uses a fixed-point complex number encoding to enable homomorphic computations on real numbers. Similar to B/FV and BGV, CKKS has operands in $R = \mathbb{Z}[X]/\langle \Phi_M(X) \rangle$, where $\Phi_M(X)$ is the cyclotomic polynomial $(x^N + 1)$ of order $M = 2N$ (cyclotomic index) and degree $N \in \mathbb{Z}$ which is the ring dimension. CKKS parameters include the ring dimension, ciphertext modulus, and standard deviation of the error, represented as (N, q, δ). To ensure at least a 128-bit security level, the ratio of N to $\log(q)$ must exceed a specific threshold value [49]. We employ the CKKS parameters to maintain 128-bit security in both classical and quantum contexts. A FHE scheme can be described by the following set of randomized algorithms:

- FHE.Setup(1^λ): Given a security parameter λ, outputs a set of encryption parameters $parms$.
- FHE.KeyGen($parms$): Outputs a secret key sk and a public key pk. Optionally, outputs one or more evaluation keys evk. If using Threshold-FHE (TFHE) functionality, on input of security parameter λ and party i, THFE.$DistSetup$(1^λ, i) outputs a share of the secret key sk_i.
- FHE.Encrypt(m, pk): Given a message $m \in \mathbb{Z}$ outputs ciphertext $c \in R$.
- FHE.Decrypt(c, sk): Given ciphertext $c \in R$, outputs message $m \in \mathbb{Z}$. If using the TFHE functionality, party i uses sk_i to output partial decryption $m : sk_i$. Then, one party combines the partial decryptions using pk and either output m or the symbol $\perp$.
- FHE.Evaluate(C,($c_1, \cdots, c_k$), evk): Given an arithmetic circuit f with k input wires, and inputs $c_1, \cdots, c_k$ with $c_i \rightarrow$ FHE.Encrypt(m_i, pk), outputs a ciphertext c such that,
  Pr[FHE.Decrypt(c, sk) $\neq f(m_1, \cdots, m_k)$] = negl(λ).

A standard security notion for FHE is indistinguishability under the chosen plaintext (IND-CPA) attack. For the threshold functionality in our protocol, we utilize TFHE, which allows for distributed decryption of ciphertexts [6]. As in standard homomorphic encryption schemes, we require that a TFHE scheme satisfies compactness, correctness, and security.

SIMD: In FHE, we can consider the factorization of $(x^N + 1)$ modulo p such that p is a prime. We can then write the message space as a direct product of small fields, encrypt a vector of elements of these fields, and operate in parallel on the entries of these vectors, thus obtaining single instruction, multiple data (SIMD) capabilities [51].

Parameters: The protocol involves $l + 1$ entities, namely $P_0, P_1, \ldots, P_{l-1}$ and P_y where, P_y is the receiver, P_{l-1} is the senders' leader and the rest of the parties are the senders. All senders possess sets of items with a bit-length of σ. The receiver holds an element y with a bit-length of σ; senders $P_0, \ldots, P_{l-1}$ own sets $X_0, \ldots, X_{l-1}$, and we define $\mathcal{X} = \bigcup_{i=0}^{l-1} X_i$.
Functionality: Inputs are encryption of y and encryptions of the sets $X_0, \ldots, X_{l-1}$. The receiver gets $\{y\} \cap \mathcal{X}$, and the senders, including the sender's leader, get $\perp$.

Fig. 1. Ideal functionality $\mathcal{F}_{PSMT}$ for PSMT for one-sided output.

C. Private Segmented Membership Test (PSMT)

For a party P_y, with a data element y, and l parties $P_0, P_1, \ldots, P_{l-1}$, each with a set X_i such that $\mathcal{X} = \sum_{i=0}^{l-1} X_i$, a PSMT allows the party P_y to learn $\{y\} \cap \mathcal{X}$, without leaking any elements not in any X_i or which party holds an element in the intersection. The sets held by different senders in the PSMT protocol are mutually exclusive or disjoint. The parties involved in the protocol are referred to as the *sender* and the *receiver*. The ideal functionality $\mathcal{F}_{PSMT}$ associated with the PSMT protocol is presented in Figure 1. It is a one-way protocol that outputs $\{y\} \cap \bigcup_{i=0}^{l-1} X_i$ to the receiver and nothing to the senders without leaking any other information about the receiver's element and sets held by the senders to any party. We note that in the event where $y \in X_i$, the set X_i is hidden from the receiver, who only learns that y is in some sender's set and not any particular set. This functionality ensures the provenance privacy of the senders. For strong protection of user data, we require that each sender P_i not have plaintext access to X_i, i.e., each sender only has encryptions of $x \in X_i$. While this could be relaxed in some cases, this requirement precludes a leak of user data in the case that a party's storage (e.g., AWS S3) is compromised.

We note that in the rest of this paper when we make reference to the senders' sets, we refer to the encrypted sets of users that are held by the senders.

IV. PSMT Protocol

A. Notation

We use λ to denote the security parameter for the FHE scheme. We use $[l]$ to denote the integers in $[0, l-1]$. Logarithms are base-two unless otherwise stated.

- $X_0, X_1, \ldots, X_{l-1}$ are the sender's set for l senders. y is the receiver's query element. Without loss of generality (WLOG), we assume X_{l-1} is the leader sender.

- c_{x_i} denote the ciphertext of the i-th sender such that $i \in [0, l-1]$. The senders' set elements are encrypted in a SIMD fashion. If $|X_i| >$ batch size of the FHE scheme, then senders can possess a set of multiple ciphertexts. c_y is the ciphertext of the receiver's element.
- N is a power of 2 and is the ring dimension in our FHE scheme; q is the ciphertext modulus; d is the homomorphic multiplicative depth of the computation.
- L, R, and n are parameters used in the application of Domain Extension Polynomials. c is the degree of polynomial used for Chebyshev polynomial approximation.
- j, k and ρ are optimization parameters to reduce the false positive rates and τ is the threshold required to confirm an intersection.

B. Basic Protocol Without Approximation

Protocols using FHE to implement PSI [11], [12], [21] use zero as a multiplicative annihilator. Multiplications are not scalable, so we propose the novel idea of using additive aggregation.

We consider set elements to be members of $\mathbb{Z}$ (e.g., hashed elements). In Figure 2, we outline our basic protocol. The receiver has access to a query element y, and s/he creates replicas of y such that the number of replicas equals the batch size of the FHE scheme and encrypts them to obtain c_y such that all slots of c_y contain y. The receiver then sends c_y to l senders. Each sender $i \in [0, l-1]$ has access to a set X_i, and they encrypt X_i using FHE batching to obtain c_{x_i} such that each slot of c_{x_i} contains different elements of X_i. Due to batching, each sender possesses only a single ciphertext. In case $|X_i| >$ batch size, the sender possesses multiple c_{x_i}. Each sender i then computes a homomorphic difference of c_y, and its ciphertext(s) c_{x_i}, diff_i in a SIMD fashion. Each sender i then computes a piecewise function $etan_i$, defined below.

$$\mathit{etan}_i(\mathit{diff}_i) = \begin{cases} K, & \text{if } \mathit{diff}_i = 0 \\ 0, & \text{if } \mathit{diff}_i \neq 0 \end{cases}$$

Namely, etan_i maps the input to K if the input is zero and zero otherwise. If equipped with multiple diff_i, the sender can simply sum the multiple etan_i into a single etan_i. Results obtained by the senders can be additively aggregated by computing $\sum_{i=0}^{l-1} \mathit{etan}_i$, whose value is non-zero for an intersection and zero for a non-intersection. If the senders' sets are disjoint, this value will be K for an intersection; otherwise, it will be some multiple of K. We note that in practical deployments, it is most probable that some leader sender will take on the additional burden of aggregation or that different senders will take turns doing this. In our protocol, a leader sender aggregates the results. Critically, the aggregation should be efficient and scale well with an increasing number of parties.

This basic protocol can be implemented with any FHE scheme, and the aggregation would be composed of efficient homomorphic additions only. Unfortunately, this version would suffer from high multiplicative depths in computing "if $\mathit{diff}_i = 0$." This leads to the need to approximate the function etan_i.

Input : Receiver provides a query element y; senders provides sets $X_0, \ldots, X_{l-1}$ such that $\mathcal{X} = \bigcup_{i=0}^{l-1} X_i$. Each set contains bit strings of length σ. K is the output of the function *etan* when input is zero. σ, and K are public parameters.
Output: Receiver outputs $\{y\} \cap \mathcal{X}$; senders outputs $\perp$.

1. **Setup**: The senders and the receiver jointly agree on a FHE scheme with the same batch size . Senders are provided a public key pk, and the receiver is given a secret key sk.
2. **Encryption**: The receiver encrypts its element y such that all slots of the resulting ciphertext c_y contain replicas of y and sends c_y to each of the senders. Each sender $i \in [0, l-1]$ encrypts their sets $X_0, \ldots, X_{l-1}$ using the public key and obtain $c_{x_0}, \ldots, c_{x_{l-1}}$. If $|X_i| >$ batch size of the scheme, then the sender has multiple ciphertexts.
3. **Intersection**: Using c_y each sender i:
   a. Homomorphically computes $\mathit{diff}_i = c_y - c_{x_i}$ and $\mathit{etan}_i(\mathit{diff}_i)$. If possessing multiple ciphertexts, the sender applies Step $3a.$ to the remaining ciphertexts and homomorphically summates them to etan_i.
   b. Sends etan_i to a leader sender, who then uses the FHE scheme to homomorphically compute
   $$z = \sum_{i=0}^{l-1} \mathit{etan}_i$$
   The leader sender returns the ciphertext z to the receiver.
4. **Result interpretation**: The receiver decrypts z using its secret key sk and outputs
   $$\text{Result} = \begin{cases} \text{Included} & z = K \\ \text{Not Included} & z \neq K \end{cases}$$

Fig. 2. Basic PSMT protocol for multiple parties

C. Novel Value Annihilating Function (VAF)

One naïve thought is to approximate the function $etan_i$ with a polynomial, but doing so would require polynomials with high degrees.

We begin by considering the computation a single sender must perform. Suppose there exists a small negligible value ϵ and a sender has a set X_i with size n such that $x_j \in X_i$ for $j \in [n]$ and is given an FHE ciphertext c_y encrypting a receiver's input $y \in Y$. Consider the function $f(y, X_i) = \sum_{j=0}^{n-1} g(y, x_j)$, where $g(y, x_j) = \frac{1}{(x_j - y) + \epsilon}$. Then, $f(y, X_i)$ can be thought to exhibit the following behavior similar to that of $etan_i$:

$$f(y, X_i) = \begin{cases} 1/\epsilon + \sum_{j=1}^{n-1} g(y, x_j), & \text{if } y \in X_i \\ \sum_{j=0}^{n-1} g(y, x_j) \quad \ll K, & \text{if } y \notin X_i \end{cases}$$

Here, when $y \in X_i$ and $\epsilon \to 0$, $f(y, X_i)$ increases without bound and its limit approaches infinity. This misuses the notion of infinity in terms of computation, as infinity is not a numerical quantity. Nevertheless, it leads us to some intuition on how to construct a protocol that is amenable to PSMT: we should have a function that each sender computes that outputs very large values (close to K) to annihilate the summation on an intersection but outputs a negligible summation value compared to K on a non-intersection. Then,

additively summing these results from each sender would yield a result indicating if the receiver's element y is in $\bigcup_{i=0}^{l-1} X_i$, i.e., the large value indicates intersections and small negligible values indicate non-intersections.

To realize this idea, we consider a candidate function $D_K^{Exact}(x) : \mathbb{R} \to \mathbb{R}$ that exhibits the useful properties described above without the problematic behavior of a potential division by zero.

$$D_K^{Exact}(x) = \begin{cases} K, & \text{if } x = 0 \\ 0, & \text{if } x \neq 0 \end{cases}$$

The function above easily satisfies these constraints and has the exact same behavior as *etan*$_i$ where $K \in \mathbb{R}$. We call such functions Value Annihilating Functions (VAFs). A VAF maps zero to K and all others to zero. Computing VAFs homomorphically is challenging because if conditions cannot be evaluated efficiently in FHE. We thus consider another function approximating $D_K^{Exact}(x)$. For such a function, approximate-arithmetic FHE is preferable, as computing such approximate functions on finite fields is difficult. It is challenging to find a function that accurately approximates such kind of behavior. The functions approximating such kind of behavior require computing the inverse or the division operation, and computing such an operation homomorphically is fundamentally challenging. Similarly, trying to control the maxima of such functions while approximation results in very bad accuracies. With all the considerations, we discovered the following function exhibits satisfactory behavior for our purpose.

$$D_{K,S}^{Approx}(x) = K \cdot (1 - \tanh^2(S \cdot (x)))$$

This function is shown in Figure 4, approximating a VAF. We need a function that is large enough if and only if the input is near zero. Piece-wise functions cannot be evaluated efficiently in FHE due to the expensive branching, so we designed a function that gets larger quickly as the input gets closer to zero. The derivative of the hyperbolic tangent $(1 - \tanh^2(x))$ has maxima as one for zero inputs and approaches zero for inputs in the range $(-\infty,-3) \cup (3,\infty)$. K acts as the maxima, and S increases the input range for zero such that the function outputs zero for inputs in range $(-\infty,-1) \cup (1,\infty)$. In particular, S adjusts the width of the function's peak or hump near the zero. Using this function for approximation, senders acquire a summation value that is equal to K when an intersection is detected; otherwise, they obtain a summation value that is negligible.

When computing functions such as $D_{K,S}^{Approx}(x)$ in approximate homomorphic encryption, only additions and multiplications are available. However, such approximations are not zero-valued for all nonzero arguments or even all arguments outside some interval centered at zero. In the following sections, we described how we compute a polynomial approximation $D_{K,S}^{Approx}(x)$ whose values on $\mathbb{R}$ except for a small interval near zero can be bounded by some small number κ. Then, we can set parameters K, S, κ, ν such that $(l+1) \cdot \kappa < \tau$, where $\tau = \nu \cdot K$. $\nu \in (0, 1]$ is a threshold proportion used to account for the case where error from approximate-arithmetic FHE may cause a sum of l outputs of $D_{K,S}^{Approx}(x)$ on nonzero arguments to be greater than $(l + 1) \cdot \kappa$.

D. Polynomial Approximation of VAF

The approximation of complex non-polynomial functions is a well-studied topic. Existing works such as [16], [17] use polynomial composition with iterative algorithms to approximate functions like min/max and comparison using FHE. Other works have used techniques like Taylor series [7], [30], minimax approximation [14], [29], [50], look-up tables [22] and conversion between FHE schemes [20].

A major challenge for approximation techniques is finding polynomial approximations that work on large domains. Existing approaches for approximation [7], [15], [29], [30] struggle for large domain intervals, inducing very high approximation errors. Even domains of thousands can be challenging [18], [29]. This would require homomorphically evaluating polynomials of an extremely large degree. For instance, simply using Chebyshev-based approximation for approximating nonlinear functions for domains such as $[-1000, 1000]$, requires almost 20.5 minutes of computation time and an additional 12 multiplicative depth for acceptable accuracy.

The error induced by the approximation and the computational cost are the major factors for approximating non-polynomial functions [18]. For our application, we are primarily concerned with the computational cost of the evaluation. Our method's requirement for approximation accuracy is low, and we only require the evaluations to yield correct approximations up to some upper bound. Specifically, we only want the evaluations to be correct so that there is a significant amount of difference in values converging to K (for intersection) and 0 (for non-intersection). More importantly, the approximation's domain interval should be large enough to enable the approximation of values in a large interval such that the sender can work with large datasets. Such approximation properties will allow the protocol to handle potentially millions of inputs.

Domain Extension Polynomials (DEPs). DEPs enable the effective shrinking of a large domain interval $[-L^n R, L^n R]$ to a smaller subinterval $[-R, R]$ such that the property of the VAFs around zero in the smaller subinterval is preserved. To compress inputs from an interval $[-L^n R, L^n R]$ to an interval $[-R, R]$, we can iteratively apply a DEP with $O(n)$ operations and $2n$ additional depth. Using this method we can convert inputs $z \in [-L^n R, L^n R]$ into values $D(z)$ such that $z \in [-R, R] \implies D(z) \approx z$ and $z \notin [-R, R] \implies D(z) \approx sign(z)$. Specifically, we utilize Algorithm 1 from [18]. To handle inputs in $[-R, R]$ using the DEP $B(z) = z - \frac{4}{27} z^3$ at an iteration n of Algorithm 1, we divide inputs to $B(z)$ by $L^n R$, and scale its ouputs by $L^n R$. This converts $B(z)$ from a DEP on $[-1, 1]$ to a DEP on $[-R, R]$. If the accuracy of $B(z)$ is not good enough, then squaring its outputs can make $B(0)$ larger and $B(z)$ for $z \neq 0$ smaller, at the cost of additional depth and runtime.

Chebyshev Approximation. Chebyshev polynomial approximation are minimax-based polynomial approximation method that achieves the smallest possible polynomial degree with minimal approximation errors. Approximating functions using standard approximation techniques like Taylor series can induce the Runge phenomenon that causes an approximation to yield poor accuracy at the edges of the interval. Chebyshev polynomials reduce this phenomenon and provide an approximation that is close to the best polynomial approximation to a continuous function. We use this method to approximate a VAF, $f(x) = 1 - \tanh^2(x)$ over the interval $[-R, R]$ using a linear combination of Chebyshev polynomials of low degrees.

We apply DEPs and Chebyshev-based approximation to approximate a VAF in a much smaller interval $[-R, R]$ while preserving the original larger domain with low FHE multiplicative depth. This leads our protocol to approximate $etan_i$ with low approximation errors and computational cost for a large domain.

E. Choosing Appropriate Parameters

Over the course of our experiments, we encountered several challenges in determining suitable parameters for the DEP and Chebyshev based approximation to tailoring them to the specific sizes of sender sets. Using $B(x) = x - \frac{4}{27}x^3$, it is imperative to maintain L below $1.5 \times \sqrt{3}$ while using DEPs [18]. Expanding the domain's range involves increasing R and n so that $[-L^n R, L^n R]$ adequately accommodates large input sets. However, using arbitrary large values for R and n would require very high depths for homomorphic computation and amplify errors in the approximation. Using smaller values of L, especially close to 1.5, greatly increases the accuracy of the DEP method, but this results in larger values for other parameters, incurring very high computational costs. Thus, finding the right balance for DEP parameters and trying to simultaneously make both the Chebyshev polynomial approximation degree and computational depth as small as possible was quite challenging. We list our parameter findings in Table I for different senders' set sizes denoted by $|\mathcal{X}|$. We note that the values described here and in Table I are useful for the case where $\bigcap_{i=1}^{l} X_i = \varnothing$, i.e., each sender's set has no element in common with other sender sets. We can allow up to Ψ of l senders to have a duplicate element, so long as values up to $\Psi \cdot K$ can be represented correctly in CKKS with the parameterization being used. We also note that these parameters that we provide are highly optimized for the particular computations that we perform, and some values in Table I were found empirically.

F. Set Updates

Even with honest senders, an attacker compromising the storage or communications of a sender may be able to learn users' private data if this data is not encrypted. Prior work in FHE-based PSI [11], [12], [21] held sender data in plaintext form. With our method, sender set elements can be provided to the senders from data owners in encrypted form, precluding this attack.

As we do not require any preprocessing of senders' sets, updates to the encrypted senders' sets are trivial in our protocol. Most computations in our protocol take place during the online phase of the protocol. This allows the senders to easily add or delete any element in their sets, as no preprocessing is necessary during the offline phase. Parties can compute the intersection of their private sets on a regular basis with sets that are often updated. In previous similar works, when parties update their sets, various preprocessing operations were essential [3], which is difficult on sets held in the encrypted form. Our protocol does not suffer from such issues.

G. Security and Correctness

Using our basic protocol in Figure 2, the receiver can learn the aggregation value $\sum_{i=0}^{l-1} etan_i$ to infer information about the difference between their query and the sender's value. The receiver can then infer about the senders' set elements not present in the intersection. To fix this issue, we require the senders to obscure the aggregation value by adding a small random masking term $r \in [-\kappa, \kappa]$, where κ is public and the bound for the approximation value of VAF on a non-intersection.The semantic security of the PSMT protocol follows directly from the IND-CPA security of the underlying FHE scheme, and we provide proof for such security in the following sections.

1) Protocol Correctness: In the protocol, if, after decryption, values in one of the slots of plaintext crosses the threshold τ, then we can say that the query element y is present in one of the senders' sets X_i. The probability of this conjecture is true except for some negligible chance that can result in a false positive. In other words, the PSMT protocol is correct if,

$$\Pr[\{y : \text{FHE.Decrypt(z)} \geq \tau\} | \{y\} \in \cup_{i=0}^{l-1} X_i] = 1 - negl(k)$$

2) Security Against Semi-Honest Adversaries: We prove the security of the protocol under the assumption of the existence of an IND-CPA secure FHE scheme Π. Here, we consider the protocol participants as corrupted by the *adversary* where they are honest but curious. Loosely put, we say that the protocol Π_{PSMT} of Figure 3 realizes the functionality of $\mathcal{F}_{PSMT}$, if it is correct.

Theorem 1. *The protocol in Figure 3 is a secure protocol for $\mathcal{F}_{PSMT}$ in the semi-honest setting.*

Proof. We consider two classes of adversaries. The first class involves adversaries that corrupt a subset of the sender parties, including the receiver P_y. The second class involves adversaries that corrupt a subset of the sender parties, not including the receiver P_y. We provide a separate simulation for each class.

Consider an adversary $\mathcal{A}$ that corrupts a strict subset $\mathcal{I}$ of parties from the set $\{P_y, P_0, \ldots, P_{l-1}\}$, including P_y. We define a simulator $\mathcal{S}$ as follows:

1. Given $\{X_i\}_{i \in \mathcal{I}}$, which also includes P_y's input, and $Z = \cap_{i=0}^{l-1} X_i$, the simulator $\mathcal{S}$ invokes the corrupted parties on their corresponding inputs.

TABLE I

PSMT PROTOCOL PARAMETERS FOR DIFFERENT SENDERS' SET SIZES WHERE K=1 AND S=10 NK: PUT THIS TABLE IN THE EXTENDED VERSION FOR ICDCS

$\lvert\mathcal{X}\rvert$	DEP			Chebyshev Parameters		Optimization			Aggregation Result	
	L	R	n	Degree	κ	j	k	ρ	Non-intersection	Intersection
2^7	2.50	21.0	2	27	3.4×10^{-5}	3	3	2.5	3.5×10^{-5}	1528.18
2^8	2.56	16.0	3	27	3.2×10^{-1}	1	3	2.5	3.3×10^{-1}	1526.54
2^{10}	2.58	24.0	4	27	1.8×10^{-6}	4	3	2.5	1.5×10^{-5}	1525.88
2^{13}	2.58	27.5	6	27	5.6×10^{-3}	4	3	2.5	5.7×10^{-3}	1525.88
2^{15}	2.58	43.5	7	27	3.7×10^{-1}	4	3	2.5	3.8×10^{-1}	1526.63
2^{20}	2.59	200.0	9	247	1.2×10^{-1}	2	3	2.5	1.2×10^{-1}	1526.11
2^{21}	2.59	400.0	9	247	7.3×10^{-1}	4	3	2.7	7.4×10^{-1}	2824.66
2^{22}	2.59	800.0	9	247	7.6×10^{-1}	6	3	2.7	7.8×10^{-1}	2824.29
2^{23}	2.59	1600.0	9	247	6.3×10^{-1}	8	3	2.7	6.5×10^{-1}	2824.00
2^{24}	2.59	3200.0	9	247	7.9×10^{-1}	10	3	2.7	8.4×10^{-1}	2824.20
2^{25}	2.59	6400.0	9	247	2.7×10^{-1}	12	3	2.7	3.0×10^{-1}	2823.92

2. $\mathcal{S}$ generates $(pk, sk) \leftarrow$ FHE.KeyGen(1^λ) by invoking the simulator's functionality $\mathcal{S}_{\text{GEN}}$ during the key generation phase.
3. Next, $\mathcal{S}$ plays the role of honest parties against P_y on arbitrary sets of inputs. Namely, $\mathcal{S}$ computes the subtractions, applies the DEP procedure, evaluates the VAF, and performs squaring and scaling. $\mathcal{S}$ then sends ciphertexts encrypting these inputs to the leader, who summates them and transmits the summated values.
4. Finally, at the concluding phase which includes the decryption and interpretation of the result, the simulator completes the decryption procedure as follows. For $y \in Z$, $\mathcal{S}$ forces the decryption outcome to cross the threshold value in one of the plaintext slots. This can be done by homomorphically adding the result ciphertext to another ciphertext that contains a value greater than τ in one of its slots. If $y \notin Z$, the simulator forces the decryption outcome to not cross the threshold value in any of the plaintext slots. This can be done by homomorphically multiplying the result ciphertext to a ciphertext encrypted with zeroes as an input. This ensures all the values of the ciphertext are less than τ.

Note that simulator $\mathcal{S}$ needs to produce simulated ciphertexts that are homomorphically evaluated and indistinguishable from the real ciphertexts because this is the only message that appears in the view of the corrupted party. This indistinguishability follows from the IND-CPA security of the underlying FHE scheme Π, since both the simulated ciphertext and real ciphertext look like fresh encryptions of the scheme Π. Hence, the view of the corrupted receiver to the ciphertext received from the simulator is indistinguishable from the receiver's view in the real execution of the protocol.

Next, we consider an adversary that does not corrupt P_y. In this case, the simulator $\mathcal{S}$ is defined as follows.

1. Given $\{X_i\}_{i \in \mathcal{I}}$, which does not include P_y's input, and $Z = \cap_{i=0}^{l-1} X_i$, the simulator invokes the corrupted parties on their corresponding inputs.
2. $\mathcal{S}$ generates $(pk, sk) \leftarrow$ FHE.KeyGen(1^λ) by invoking the simulator's functionality $\mathcal{S}_{\text{GEN}}$ during the key generation phase.
3. Next, $\mathcal{S}$ plays the role of P_y against the corrupted senders on an arbitrary set of inputs and concludes the simulation by playing the role of P_y on these arbitrary inputs. This corruption case is simpler as only P_y learns the output.

In this case, where we have only corrupt senders, the simulator $\mathcal{S}$ can simply generate new encryptions of zero in the place of real encryptions of the receiver's query element y. By the IND-CPA security of the FHE scheme, this ciphertext is indistinguishable from the senders' view in the real protocol. This concludes the proof for the second case.

□

The parties are semi-honest in PSMT. Hence, they are guaranteed to use the actual inputs; therefore, there is no integrity tampering. Unlike previous PSI protocols, our PSMT protocol directly operates on the encrypted sets; therefore, the senders are also assumed to be semi-honest. We do not require an extra pre-processing step such as OPRF to guarantee the privacy of the set against the senders [11]. This enables us to avoid the expensive offline preprocessing step and also makes the update in the sender's set straightforward and feasible.

H. Asymptotic Complexity Analysis

Each sender must compute the procedures described in Figure 3. This requires $2n + O(log(c)) + j + k$ homomorphic multiplications. These procedures require a multiplicative depth of n for the application of the DEP, approximately $log(c) + 1$ for Chebyshev polynomial approximation, and $j + k$ for repeated squaring. The precise amount of depth recommended for a polynomial approximation of degree c is chosen heuristically[1]. CKKS operations, besides multiplication, e.g., addition and scaling, contribute relatively small amounts of overhead and noise. The additive aggregation of each sender's result is relatively inexpensive; most notably, it does not increase multiplicative depth *even with an increasing number of senders*.

Our protocol requires one ciphertext to be sent from the receiver to the l senders. Then, the senders compute their individual results and then forward them to an aggregator. After aggregation, the result is forwarded to the α parties holding TFHE keys if TFHE is being used. After performing partial decryption, those parties then return their result to the

[1]Some guidelines for choosing parameters for polynomial approximation can be found at https://github.com/openfheorg/openfhe-development/blob/main/src/pke/examples/FUNCTION_EVALUATION.md

TABLE II
OVERALL COMPLEXITY OF OUR PROTOCOL. S-TO-S DENOTES THE INTER-SENDER COMMUNICATION

Protocol	Communication R-to-S —— S-to-R —— S-to-S	Computation	# Homomorphic Mult.
PSMT	$O(l)$ —— $O(\alpha)$ —— $O(l \cdot \alpha)$	$O(n + \sqrt{c} + j + k)$	$O(n + \log^2 c + j + k)$

receiver, who aggregates them to complete decryption. Thus, the communication between the receiver and other parties is bounded above by $l + \alpha$ ciphertexts, and the inter-sender communication is bounded above by $l \cdot \alpha$ ciphertexts. There are three communication rounds between the receiver and senders if TFHE is not used. If TFHE is implemented, there is an extra round of communication for the partial decryption by the keyholding senders. Table II shows PSMT's overall complexity.

Tradeoffs: The process of applying a DEP followed by Chebyshev polynomial evaluation requires significant depth, but this depth does not depend on the number of senders involved. Each sender can perform this computation in parallel, but this step may be expensive. Thus, even though we require a higher computational complexity compared to other methods for a small number of senders, this complexity does not depend on the number of senders and stays moderately low for a significantly higher number of senders. The protocol's computation and FHE multiplicative depth only depend on the size of the sender's set and not the number of senders. We also note that FHE batching can only be applied to either the receiver or senders, but not both. Applying batching to the receiver's elements will require using hashing and binning, which will eventually require pre-processing of the elements. Hashing and partitioning are applied in previous works to divide the sender's set into smaller, more manageable subsets [12], [21]. With partitioning, the sender's set is simply partitioned into α subsets of approximately equal size, which reduces network communication and computation by a factor of α. Hashing similarly divides elements into hash buckets, and the parties only need to compare elements in the same bucket. Both optimizations can be applied to our PSMT protocol; it will require pre-processing and increase the input domain, significantly affecting the computational overhead and lowering the privacy level of datasets held by the senders.

V. CHALLENGES

A. Increasing Throughput

Using our PSMT protocol, senders encode each of its sets' elements into separate plaintexts and individually compute the polynomial approximation. A single query can take time on the order of seconds, highlighting the need for better throughput. The throughput of our PSMT protocol can be improved using the SIMD feature available in FHE schemes. The CKKS scheme supports the packing of multiple message vectors into a single ciphertext where the slots of the ciphertexts hold different values. This allows slot-wise addition and multiplication [51]. Using this mapping, the CKKS scheme can encrypt $b = N/2$ elements in $\mathbb{R}$ into a single ciphertext.

We apply batching to reduce the protocol's computational and communication costs. The receiver encodes their query y into a single ciphertext c_y, and the sender encodes their multiple values $x_i \in X$ in a single ciphertext c_{x_i} with the same batch size used by the receiver. If a sender's set size is greater than the batch size, then multiple ciphertexts are used and evaluated in parallel. Each sender i then homomorphically computes $z = \sum_{i=0}^{b-1} 1 - \tanh^2(c_y - c_{x_i})$, and sends z back to the receiver. The batching technique allows the sender to operate on b items simultaneously, resulting in b-fold improvement in both the computation and communication. We note that FHE batching can only be applied to at most one of the receiver or the senders. To enable batching when not all slots are used, we fill the remaining slots with a dummy value, which is negligible and does not alter the intersection results.

B. Supporting Larger Sets

Increasing the capacity of the protocol to support large senders' sets requires the use of larger plaintext spaces. In prior work on FHE-based PSI [11], [12], [21], the sender partitions their elements into disjoint sets and computes the intersection of receiver elements with each partition. This can reduce the effective set size in each computation, reducing the multiplicative depth that the FHE parameters must accommodate. However, the result from each partition must be sent back to the receiver, increasing communication overhead. Theoretically, the results from each partition could be multiplicatively aggregated to a single response, but this will require even more multiplicative depth. An advantage of our design is that we gain the ability to partition computations and thereafter aggregate results from the partitions nearly for free. Homomorphic addition is a depth-free operation [15], and doing so when a single sender has multiple ciphertexts does not require additional depth.

Our protocol can be extended to use very large sender sets. However, we note that it will require more depth and computing power due to the large FHE parameters required for the DEP procedure and Chebyshev approximation. Since our final result is the summation of individual results from the senders, increasing the senders' set sizes does not dramatically increase the communication size.

C. Reducing False Positives

In the existing PSI-related works, false positives occur due to hashing collisions or representation of the set elements using some probabilistic encoding techniques like bloom filter encoding. In our protocol, false positives can occur due to bad approximation accuracy of the VAF. After applying the DEP procedure, we found that the homomorphic approximation for the VAF was not accurate enough within the range $[-3, 0] \cup [0, 3]$. While the approximation would correctly map zero values to K, the non-zero values in the range $[-3, 0] \cup [0, 3]$ would be mapped to values far too close to K. Hence, instead of producing a hump at 0, we observed that the approximation would produce a flatter curve. These values would later contaminate the summation outcome and cause

the result to cross the threshold in cases of no intersection, thereby leading to false positives. To solve this issue, we used a technique involving homomorphic squaring and scaling. First, we forced this mapping of values in the range $[-3, 0] \cup [0, 3]$ to fall in the range $(-1, 0) \cup (0, 1)$ by setting $K = 1$. After this step, we repeatedly applied homomorphic squaring to these results until they fell into $(-0.5, 0) \cup (0, 0.5)$. Note that squaring would not affect the value at 0 as it remains mapped to 1. Next, we scaled up all the values by a scaling factor ρ such that the values in $(-0.5, 0) \cup (0, 0.5)$ were mapped within $(-1, 0) \cup (0, 1)$. We discovered that selecting $\rho \in (2, 2.8)$ sufficed. Finally, we apply homomorphic squaring to the values again so that the difference between the value mapped to one and values in the range $(-0.5, 0) \cup (0, 0.5)$ are increased by the square of the factor ρ^2. Note that squaring after scaling would only map the non-intersection values to smaller and smaller values, as they are all less than one. After using the aforementioned techniques, κ is the highest value approximated by the Chebyshev when diff_i (see Figure 3) is at its minimum value of one (Table I). We can see that values of κ are very close to the aggregation result of non-intersection values, indicating that the differences larger than one are mapped to negligibly small numbers.

D. Decentralizing Data Security

In prior work using FHE for privately computing set functions, the receiver holds a FHE secret key. Even though the receiver is able to decrypt any ciphertext, security is maintained through data isolation: the sender never sends any encryption of information besides a final result to the receiver. In PSMT, we are able to protect data at rest by encrypting datasets held by senders. However, if the encrypted dataset was leaked, a receiver holding the FHE secret key would be able to decrypt this dataset.

To preclude this vulnerability, we propose to use threshold FHE (TFHE). During setup, instead of giving an FHE secret key to the receiver, a trusted setup procedure (which can be run via trusted hardware or secure multiparty computation) will distribute shares of the TFHE secret key to φ parties, which may be any of the senders, receiver, or other parties. After the senders' results have been homomorphically summed, the resulting ciphertext z is broadcast to the φ key shareholders. Each of these parties performs partial decryption and then sends their result to the receiver, which combines the partial decryptions to get the final result. The φ secret key shareholders must be trusted not to collude with each other to share the partial decryptions, as any party who learns all partial decryptions can combine themselves to learn the result. The receiver can be given one of the TFHE secret keys to accomplish this.

Our complete protocol after incorporating the aforementioned solutions is detailed in Figure 3.

VI. Evaluation

We implemented our proposed PSMT protocol using C++17 and OpenFHE v1.0.3. The anonymized source code is available at https://anonymous.4open.science/r/psmt-1323/. We use the CKKS encryption scheme [15] and OpenFHE's default parameter settings to ensure 128-bit security ($\lambda = 128$). Our experiments were run on a server with an AMD EPYC 7313P processor and 128GB of memory, running Ubuntu 20.04.

We evaluated the latency of per-sender query runtime and the runtime for aggregating results from multiple senders by performing 5 PSMT runs for each combination of parameters and taking the average. The bit-length δ of the set elements is matched to the plaintext space accommodated by the DEP and Chebyshev parameters. The senders' individual computations are independent in our protocol. We assume that after receiving the query ciphertext from the receiver, each sender operates on their own set in parallel. We choose the number of senders for every senders' set size from 2^7 to 2^{25} such that each sender has to evaluate a single ciphertext, i.e., a single sender possesses at most of the batch-size amount elements. In case we have fewer senders for a given $|\mathcal{X}|$, which results in a single sender possessing more than batch-size elements, we assume that senders pack their set elements in multiple ciphertexts and evaluate them parallelly and homomorphically summates them to a single ciphertext. This ensures that query computation time is not affected due to multiple evaluations of the ciphertexts.

Baselines. Many variants of PSI, MPSI, and PMT protocols are designed to handle specific scenarios or more general scenarios. Selecting a PSI, MPSI, or PMT protocol for comparison with our work is a complex task, given our novel privacy model with server-side encryption and a high number of sender parties. Considering the existing works discussed in **??**, [2] and [48] either lack scalability for a large number of senders or do not provide a public implementation. [46] and [9] both lack post-quantum security. [12] is an older work and serves as the foundation for [11]. Thus, we compare our protocol to the following state-of-the-art baseline PSI and MPSI protocols: Cong *et al.*'s PSI [21] and Bay *et al.*'s MPSI [4]. Cong *et al.*'s PSI is based on [11], which uses the BFV scheme [24]. It is the state-of-the-art PSI protocol based on FHE with post-quantum security. For a fair comparison, we implemented their protocol for a multiparty setting using OpenFHE (based on C++) and their public implementation, which was implemented using the SEAL library. Bay *et al.*'s MPSI is inspired by the Davidson-Cid protocol based on Bloom filters, and it does not use FHE. It is among the fastest-known MPSI protocols for smaller set sizes and a large number of parties, making it suitable to compare with our work. We used their public implementation (also based on C++) for the following experimental comparison.

Table III shows query computation time, which is the total time required to complete the DEP and Chebyshev approximation using the parameters provided in Table I. For these experiments, the senders' set size is the total input domain for the input entries present in the sets. We assume that each input in the set is identifiable using a unique hash-ID commonly used in PSI applications (SHA-256). Observe that the size of messages from the receiver to senders increases

TABLE III
COMMUNICATION AND COMPUTATION OVERHEAD FOR PSMT. R-TO-S AND S-TO-R DENOTE THE RECEIVER-TO-SENDERS AND SENDER-TO-RECEIVER MESSAGE SIZE, RESPECTIVELY. AGGREG. DENOTES THE TOTAL TIME REQUIRED TO AGGREGATE THE CIPHERTEXTS FOR THE CORRESPONDING NUMBER OF SENDERS. QUERY REFERS TO THE QUERY COMPUTATION TIME FOR SENDERS.

$\|\mathcal{X}\|$	**Message size (MB)**		**Senders**	**Aggreg.(s)**	**Query (s)**	**Depth**
	R-to-S	S-to-R				
2^7	43	4.1	128	6.53	13.20	21
2^8	49	4.1	256	14.12	15.24	23
2^{10}	63	4.1	1024	40.29	19.09	30
2^{13}	79	4.1	1024	67.51	26.00	38
2^{15}	87	4.1	1024	71.16	30.07	42
2^{20}	107	4.1	1024	87.78	59.67	52
2^{21}	111	4.1	1024	103.98	64.65	54
2^{22}	115	4.1	1024	84.63	68.90	56
2^{23}	119	4.1	1024	90.34	75.80	58
2^{24}	123	4.1	1024	84.09	81.08	60
2^{25}	127	4.1	1024	92.53	86.76	62

with senders' set sizes due to larger senders' set size requiring query ciphertexts, which should be able to tolerate a higher amount of noise and, hence, higher depth for the DEP and Chebyshev. The message size from the senders to the receiver remains constant in our protocol as it only contains the summation result ciphertext in which the slots hold the intersection information. For various senders' set sizes $|\mathcal{X}|$, we set the highest number of parties to 1024 (modest for most applications with a high number of parties) and examine the aggregation latency and communication for larger sets. Most related state-of-the-art work is only evaluated for a small number of senders (usually 100 or less) [4], [5], [32]. Notably, even for 1024 senders, the aggregation runtime (not counting encryption/decryption) of PSMT stays within 100 seconds for senders' set sizes up to 2^{25}. Our protocol can be easily extended to a larger number of senders and senders' set sizes. In our current set of parameter configurations for senders' set sizes up to 2^{25}, we show that we can handle up to 4096 parties with a false positive rate of zero (Figure 6). We note that we only report computational latency for only up to 4096 parties, but this is only due to memory constraints when running the protocol with a huge number of senders in a single system. We also applied parallel computation while aggregating the ciphertexts to obtain a superior runtime when compared to sequential aggregation of the ciphertexts.

Comparison with Cong *et al.* [21]. We compare the computational latency of PSMT and Cong *et al.*'s protocol in Figures 5 to 7. For the number of senders less than 150, Cong *et al.* 's protocol is faster than PSMT, but for higher senders, PSMT outperforms Cong *et al.* (Figure 5). As the number of senders increases, aggregation time for Cong *et al.* becomes the bottleneck due to *multiplicative* aggregation. We use OpenFHE's built-in method *EvalMultMany()* while aggregating ciphertext for Cong *et al.* that uses a binary tree to keep the depth growth required for aggregation to be logarithmic. Using this mechanism for aggregation still induces a high number of homomorphic multiplications, resulting in a high computation time. For PSMT, the required depth for aggregation for a higher number of senders remains *constant* since it relies on *additive* aggregation.

In Figure 7, we can observe the computational latency for Cong *et al.* and PSMT. Cong *et al.* 's query computation time is in order of milliseconds; however, their protocol has a huge aggregation overhead. We can observe the same phenomenon in Figure 6 for a higher number of senders and a fixed senders' set size. The query computation time for PSMT is slower than Cong *et al.* 's, but PSMT is almost 11x faster when the aggregation time is also taken into account for the overall computational latency. In terms of communication, Cong *et al.* 's protocol is better than PSMT. PSMT, however, operates on encrypted senders' sets, unlike Cong *et al.* , where senders' sets are needed in plaintext for initialization steps like polynomial interpolation. This also makes their scheme undesirable for applications that require frequent database updates but, updates can be easily done in PSMT. Both protocols have negligible false positive rates when proper scheme parameters are employed.

Comparison with Bay *et al.* [4]. We found this work closest to our PSMT protocol in the MPSI setting. Their protocol targets scenarios where many parties want to compute the collection of items that a leader shares with at least β parties. In PSMT, β is set to one by definition since all senders have unique elements, and only one intersection is possible. We compare the runtime for their protocol when β is set to one while running the experiments. It is important to note that their protocol induces a false positive probability of 1% due to bloom filters, while PSMT has no false positives. In their protocol, senders can have duplicate elements among each other, but in PSMT, we assume that all senders have distinct elements. Hence, for the number of parties higher than 128 and 256 in PSMT and Cong *et al.* , we use the senders' set size of 2^8 and 2^9, respectively, even though this results in higher computational latencies for the two protocols. Finally, Bay *et al.* , like other PSI and MPSI protocols, accesses the senders' set element in plaintext, while PSMT operates on encrypted sets. The runtime comparison result of PSMT, Cong *et al.* and Bay *et al.* with Bay *et al.* having a senders' set size of 2^7 is presented in Figure 5. Note we report both the query computation and aggregation time for PSMT. Cong *et al.* and Bay *et al.* are more efficient than PSMT for parties less than 50 and 150, respectively. PMST is up to $4\times$ and $5.5\times$ faster than Bay *et al.* and Cong *et al.* respectively for 500 senders.

Comparison with other works. Another PMT work closely aligned with ours is the work by Tamrakar *et al.* , called Carousal [52]; however, it is a TEE-based PMT protocol that primarily targets malware detection applications and induces a false positive rate of 2^{-10}. which is very high for the application scenario our protocol targets. This setup aligns well with their application context, which involves malware detection where the data's intrinsic value is relatively low; however, in the scenario we consider, the server's data can be very sensitive (watchlisted individuals or fraud clients) and of value to some individuals. [32] is close to our work in the MPSI setting; however, it scales very poorly for a large number of parties, taking almost 300 seconds of computation for only 15 parties. We do not compare our work against [32] since it

has already been compared against [4] and [39]. [5] provide a maliciously secure MPSI based on based on garbled bloom filters and k-out-of-N OT. [5] has been compared against [39] hence we do not compare against [5]. [13] employ a multi-query reverse private membership test (mqRPMT) protocol for various private set operations, including PSI, where the server learns the intersection result of a query element. Their protocol's construction is fundamentally different from ours. Also, their public implementation is only compatible with two parties; hence we do not compare against [13]. [55] achieve maliciously secure multiparty PSI using bloom filters for large inputs; however, do not provide a public implementation.

In contrast to previous works, the computational cost of query in our protocol is not directly dependent upon the number of parties involved. Instead, it only depends on the maximum senders' set size, and the computational cost growth is much slower with the increasing senders' set size and the number of senders. We also note that the overhead of sending and receiving queries through a channel is not considered in the latencies we describe here. Prior work considered a LAN setting with a 10Gbps connection [4], [32]; in this case, our protocol can take advantage of the high available bandwidth to be able to handle a large number of senders.

VII. Conclusion

In this work, we introduce the concept of a Private Segmented Membership Test (PSMT) for cases where users wish to query a set held piecemeal among many data holders. We show a basic PSMT protocol based on approximate-arithmetic fully homomorphic encryption and provide details about overcoming various technical challenges to make our solution feasible. Our method protects the values of senders' elements from even the senders and allows for preprocessing-free updates. Our experiment shows the scalability of our protocol that aggregates the penultimate results from data holders. In future work, we aim to explore new avenues for further optimizations and better communication overhead in PSMT.

Acknowledgment

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression "one of us (R. B. G.) thanks ...". Instead, try "R. B. G. thanks...". Put sponsor acknowledgments in the unnumbered footnote on the first page.

References

Please number citations consecutively within brackets. The sentence punctuation follows the bracket . Refer simply to the reference number, as in —do not use "Ref. " or "reference " except at the beginning of a sentence: "Reference was the first ..."

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors' names; do not use "et al.". Papers that have not been published, even if they have been submitted for publication, should be cited as "unpublished" . Papers that have been accepted for publication should be cited as "in press" . Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation .

References

[1] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, et al. Homomorphic encryption standard. *Protecting privacy through homomorphic encryption*, pages 31–62, 2021.

[2] Saikrishna Badrinarayanan, Peihan Miao, Srinivasan Raghuraman, and Peter Rindal. Multi-party threshold private set intersection with sublinear communication. pages 349–379. Springer, 2021.

[3] Saikrishna Badrinarayanan, Peihan Miao, and Tiancheng Xie. Updatable private set intersection. *Proceedings on Privacy Enhancing Technologies*, 2022(2), 2022.

[4] Aslı Bay, Zekeriya Erkin, Jaap-Henk Hoepman, Simona Samardjiska, and Jelle Vos. Practical multi-party private set intersection protocols. *IEEE Transactions on Information Forensics and Security*, 17:1–15, 2021.

[5] Aner Ben-Efraim, Olga Nissenbaum, Eran Omri, and Anat Paskin-Cherniavsky. Psimple: Practical multiparty maliciously-secure private set intersection. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pages 1098–1112, 2022.

[6] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter MR Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In *Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I 38*, pages 565–596. Springer, 2018.

[7] Joppe W Bos, Kristin Lauter, and Michael Naehrig. Private predictive analysis on encrypted medical data. *Journal of biomedical informatics*, 50:234–243, 2014.

[8] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.

[9] Nishanth Chandran, Nishka Dasgupta, Divya Gupta, Sai Lakshmi Bhavana Obbattu, Sruthi Sekar, and Akash Shah. Efficient linear multiparty psi and extensions to circuit/quorum psi. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1182–1204, 2021.

[10] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H Lai. Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 142–157. IEEE, 2019.

[11] Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled psi from fully homomorphic encryption with malicious security. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1223–1237, 2018.

[12] Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1243–1255, 2017.

[13] Yu Chen, Min Zhang, Cong Zhang, Minglang Dong, and Weiran Liu. Private set operations from multi-query reverse private membership test. *Cryptology ePrint Archive*, 2022.

[14] Jung Hee Cheon, Jinhyuck Jeong, Joohee Lee, and Keewoo Lee. Privacy-preserving computations of predictive medical models with minimax approximation and non-adjacent form. In *Financial Cryptography and Data Security: FC 2017 International Workshops, WAHC, BITCOIN, VOTING, WTSC, and TA, Sliema, Malta, April 7, 2017, Revised Selected Papers 21*, pages 53–74. Springer, 2017.

[15] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong*

Kong, China, December 3-7, 2017, Proceedings, Part I 23, pages 409–437. Springer, 2017.

[16] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. Efficient homomorphic comparison methods with optimal complexity. In *Advances in Cryptology–ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II 26*, pages 221–256. Springer, 2020.

[17] Jung Hee Cheon, Dongwoo Kim, Duhyeong Kim, Hun Hee Lee, and Keewoo Lee. Numerical method for comparison on homomorphically encrypted numbers. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 415–445. Springer, 2019.

[18] Jung Hee Cheon, Wootae Kim, and Jai Hyun Park. Efficient homomorphic evaluation on large intervals. *IEEE Transactions on Information Forensics and Security*, 17:2553–2568, 2022.

[19] Eduardo Chielle, Homer Gamil, and Michail Maniatakos. Real-time private membership test using homomorphic encryption. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1282–1287. IEEE, 2021.

[20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[21] Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled psi from homomorphic encryption with reduced computation and communication. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1135–1150, 2021.

[22] Jack LH Crawford, Craig Gentry, Shai Halevi, Daniel Platt, and Victor Shoup. Doing real work with fhe: the case of logistic regression. In *Proceedings of the 6th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 1–12, 2018.

[23] Sumit Kumar Debnath and Ratna Dutta. Provably secure fair mutual private set intersection cardinality utilizing bloom filter. In *International Conference on Information Security and Cryptology*, pages 505–525. Springer, 2016.

[24] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, 2012.

[25] Michael J Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In *TCC*, volume 3378, pages 303–324. Springer, 2005.

[26] Michael J Freedman, Kobbi Nissim, and Benny Pinkas. Efficient private matching and set intersection. In *International conference on the theory and applications of cryptographic techniques*, pages 1–19. Springer, 2004.

[27] Johannes Götzfried, Moritz Eckert, Sebastian Schinzel, and Tilo Müller. Cache attacks on intel sgx. In *Proceedings of the 10th European Workshop on Systems Security*, pages 1–6, 2017.

[28] Irina Ivanova. Rich americans hide "billions" offshore thanks to tax loophole, senate panel finds, https://www.cbsnews.com/news/tax-evasion-billions-offshore-fatca-tax-reporting-loophole-senate-finance-committee-robert-brockmann/ Aug 2022.

[29] Tanveer Khan, Alexandros Bakas, and Antonis Michalas. Blind faith: Privacy-preserving machine learning using function approximation. In *2021 IEEE Symposium on Computers and Communications (ISCC)*, pages 1–7. IEEE, 2021.

[30] Miran Kim, Yongsoo Song, Shuang Wang, Yuhou Xia, Xiaoqian Jiang, et al. Secure logistic regression based on homomorphic encryption: Design and evaluation. *JMIR medical informatics*, 6(2):e8805, 2018.

[31] Vladimir Kolesnikov and Ranjit Kumaresan. Improved ot extension for transferring short secrets. In *Advances in Cryptology–CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part II*, pages 54–70. Springer, 2013.

[32] Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu. Practical multi-party private set intersection from symmetric-key techniques. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1257–1272, 2017.

[33] Anunay Kulshrestha and Jonathan Mayer. Identifying harmful media in {End-to-End} encrypted communication: Efficient private membership computation. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 893–910, 2021.

[34] Eyal Kushilevitz and Rafail Ostrovsky. Replication is not needed: Single database, computationally-private information retrieval. In *Proceedings 38th annual symposium on foundations of computer science*, pages 364–373. IEEE, 1997.

[35] Catherine Meadows. A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In *1986 IEEE Symposium on Security and Privacy*, pages 134–134. IEEE, 1986.

[36] Atsuko Miyaji, Kazuhisa Nakasho, and Shohei Nishida. Privacy-preserving integration of medical data: a practical multiparty private set intersection. *Journal of medical systems*, 41:1–10, 2017.

[37] Daniel Morales, Isaac Agudo, and Javier Lopez. Private set intersection: A systematic literature review. *Computer Science Review*, 49:100567, 2023.

[38] Muhammad Haris Mughees and Ling Ren. Vectorized batch private information retrieval. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 437–452. IEEE, 2023.

[39] Ofri Nevo, Ni Trieu, and Avishay Yanai. Simple, fast malicious multiparty private set intersection. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1151–1165, 2021.

[40] John Newman and Ritchie Amy. Financial privacy rule, Nov 2023.

[41] Department of Homeland Security. Dhs use cases of privacy enhancing technologies, June 2022.

[42] Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. Phasing: Private set intersection using permutation-based hashing. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 515–530, 2015.

[43] Benny Pinkas, Thomas Schneider, Oleksandr Tkachenko, and Avishay Yanai. Efficient circuit-based psi with linear communication. In *Advances in Cryptology–EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19–23, 2019, Proceedings, Part III 38*, pages 122–153. Springer, 2019.

[44] Benny Pinkas, Thomas Schneider, Christian Weinert, and Udi Wieder. Efficient circuit-based psi via cuckoo hashing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 125–157. Springer, 2018.

[45] Benny Pinkas, Thomas Schneider, and Michael Zohner. Faster private set intersection based on {OT} extension. In *23rd {USENIX} Security Symposium ({USENIX} Security 14)*, pages 797–812, 2014.

[46] Benny Pinkas, Thomas Schneider, and Michael Zohner. Scalable private set intersection based on ot extension. *ACM Transactions on Privacy and Security (TOPS)*, 21(2):1–35, 2018.

[47] The Washington Post. Foreign banks to help u.s. fight tax evasion, Apr 2023.

[48] Sara Ramezanian, Tommi Meskanen, Masoud Naderpour, Ville Junnila, and Valtteri Niemi. Private membership test protocol with low communication complexity. *Digital Communications and Networks*, 6(3):321–332, 2020.

[49] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 173–187, 2022.

[50] Jason Schlessman. *Approximation of the sigmoid function and its derivative using a minimax approach*. PhD thesis, Lehigh University, 2002.

[51] Nigel P Smart and Frederik Vercauteren. Fully homomorphic simd operations. *Designs, codes and cryptography*, 71:57–81, 2014.

[52] Sandeep Tamrakar, Jian Liu, Andrew Paverd, Jan-Erik Ekberg, Benny Pinkas, and N Asokan. The circle game: Scalable private membership test using trusted hardware. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, pages 31–44, 2017.

[53] Zhusheng Wang, Karim Banawan, and Sennur Ulukus. Multi-party private set intersection: An information-theoretic approach. *IEEE Journal on Selected Areas in Information Theory*, 2(1):366–379, 2021.

[54] ACC Yao. How to generate and exchange secrets. in—27th annual symposium on foundations of computer science, 1986.

[55] En Zhang, Feng-Hao Liu, Qiqi Lai, Ganggang Jin, and Yu Li. Efficient multi-party private set intersection against malicious adversaries. In *Proceedings of the 2019 ACM SIGSAC conference on cloud computing security workshop*, pages 93–104, 2019.

Parameters: X_i is the input set for l senders such that $i \in [0, l-1]$, y is the receiver's input. τ is the threshold value, and κ is the limit for the random mask. λ denotes the computational security parameter, and set elements are of bit strings of length σ. λ, τ, κ and σ are public parameters. K and S are public VAF parameters. L, R and n are public DEP parameters. c is the degree of the polynomial used for Chebyshev approximation.

1. [**Parameters**]
   a. **FHE**: Parties agree on parameters (N, q, δ, d) for the CKKS FHE scheme.
   b. **Key Distribution**: If using the threshold FHE functionality, a trusted setup provides each key shareholder $i \in [0, \varphi)$ a share of the secret key sk as one of $\{sk_0, sk_1, \ldots, sk_{\varphi-1}\}$, and broadcast the public key pk. If threshold FHE is not being used, the functionality can simply generate a public key pk for the senders and give a secret key sk to the receiver.
2. [**Encryption**]
   a. **Encrypt X & batch** : For all $x \in X_i$, such that $i \in [0, l-1]$, each sender i groups its values x into vectors in $\mathbb{R}^m$ of length m with elements in $\mathbb{R}$. Then, sender i batches each vector into $2 \cdot m/N$ plaintexts and encrypts them using pk. As a result, each slot of the resulting ciphertext contains individual x values. Each sender encrypts and batches their sets independently and possesses ciphertext c_{x_i} after this step. In case the $|X_i|$ ¿ batch-size, the sender possesses multiple ciphertexts encrypting X_i. Note that FHE batching can only be applied to either the receiver or senders, but not both.
   b. **Encrypt replicas of y**: Receiver constructs a vector of length $N/2$, with each element being $y \in \mathbb{R}$. Then, the receiver encodes the vector into a CKKS plaintext and encrypts it. As a result, each slot of the resulting ciphertext c_y contains y.
4. [**Compute Intersection**]
   a. **Homomorphically compute subtractions**: Each sender i receives the ciphertext c_y and computes $\mathit{diff}_i = c_y - c_{x_i}$.
   b. **Apply DEP to reduce the domain size**: Each sender i iteratively applies the domain extension process n times using DEP to shrink the domain interval of values diff_i from step $4a$ into the interval $[-R, R]$.
   c. **Homomorphically evaluate VAF**: Each sender i uses the Chebyshev approximation method to approximate $\mathit{etan}_i = K \cdot (1 - \tanh^2(S \cdot \mathit{diff}_i))$ in the interval $[-R, R]$ using a degree c polynomial.
   d. **First homomorphic squaring**: Each sender i homomorphically squares etan_i and obtains $\mathit{etan}_i^{2^j}$ such that $j \in [1, 12]$. Higher values of $j \geq 2$ can be chosen depending on the desired approximation accuracy for larger senders' set sizes.
   e. **Scaling and second homomorphic squaring**: Each sender i multiplies $\mathit{etan}_i^{2^j}$ using a scaling factor, ρ. Step $4d$ can be applied again to obtain $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$ with $k \in [3, 4]$ to exponentially increase the number of parties the protocol can handle and reduce the false positive rates to negligible or zero.
   f. **Homomorphically evaluate summation**: If a sender possesses multiple ciphertexts, Step $4a$ to $4e$ is applied to all the remaining ciphertexts in parallel, and the sender homomorphically summates them to $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$. An aggregator collects $(\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$ from all other senders, samples a random non-zero plaintext element $r \in [-\kappa, \kappa]$; and homomorphically evaluates

$$z = r + \sum_{i=0}^{l-1} (\rho \cdot \mathit{etan}_i^{2^j})^{2^k}$$

   and optionally performs modulus switching on the ciphertext z to reduce its size. z is then broadcast to other senders.
5. [**Decrypt and get result**]
   a. **TFHE partial decryption**: If using the TFHE functionality, upon receiving z, each key share holder $i \in [0, \varphi)$ partially decrypts the message to obtain part_i using its share of the secret key sk_i and sends part_i to the receiver.
   b. **TFHE final decryption**: The receiver decrypts the ciphertext z and obtains a resulting vector of length $N/2$. If using TFHE, the receiver combines the partial descriptions received from the φ senders, $\{\mathit{part}_i : sk_i\}_{i \in [0, \varphi)}$ using pk and its share of the secret key sk_i and obtains the resulting vector of length $N/2$.
   c. **Return result**: If TFHE is not being used, then the aggregator returns z to the receiver.
   d. **Interpretation of result**: The receiver decrypts z (either with basic FHE decryption or by combing TFHE partial descriptions), and checks if the magnitude of any element of the resulting vector exceeds τ,

$$\{y\} \cap \bigcup_{i=0}^{l-1} X_i = \{y : \text{FHE.Decrypt(z)} \geq \tau\}$$

   If any value in a ciphertext slot is greater than the threshold τ, an intersection exists; otherwise, no intersection exists.

Fig. 3. Full PSMT protocol

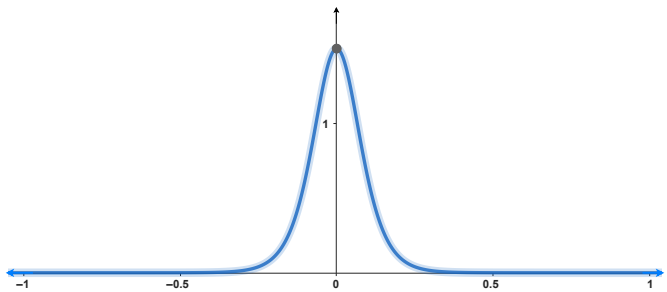

Fig. 4. Graph of the VAF $K \cdot (1 - \tanh^2(S \cdot x))$ where $K = 1.5$ and $S = 10$

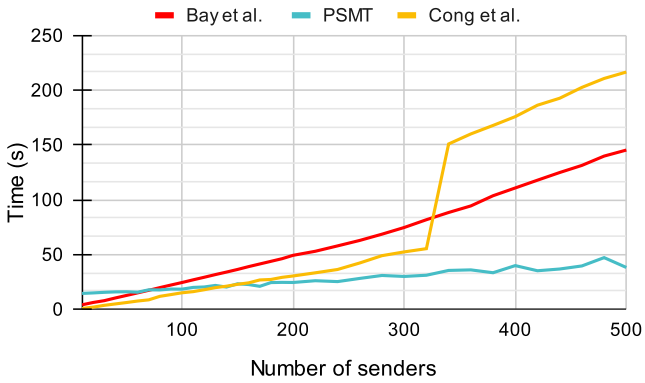

Fig. 5. Runtime comparison for Bay *et al.*'s MPSI [4] protocol with PSMT and Cong *et al.* [21]

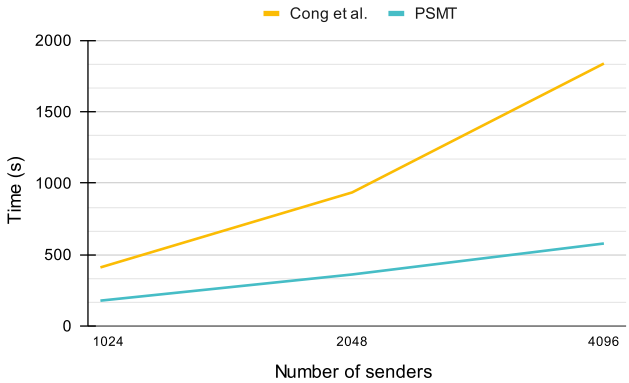

Fig. 6. Total computation time comparison for Cong *et al.* [21] vs. PSMT for different numbers of senders up to 4096. Senders' set size is set to 2^{25}.
** Bay *et al.* 's omitted due to inefficiently for set size of 2^{25}.

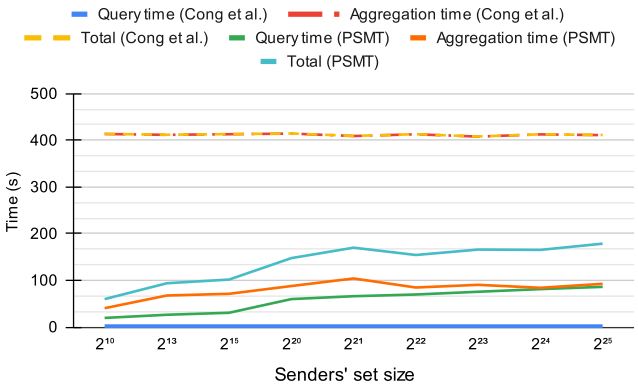

Fig. 7. Computation time comparison for Cong *et al.* [21] vs PSMT for different senders' set size. Number of senders is set to 1024.
**Bay *et al.* 's omitted due to inefficiently for larger set sizes.